

A Systematic Literature Review Protocol for Testing and Regression Testing of Quantum Software Compilers

Furqan Nasir, Arif Shah, Iftikhar Alam

Department of Computer Science, City University of Science & Information Technology, Peshawar, Pakistan

Abstract

Quantum software development kits such as Qiskit change constantly. The transpiler, the part that rewrites a circuit for real hardware, changes the most. A change to one compilation pass can introduce a regression: the compiled circuit may give a wrong result, stay correct but grow larger, or simply take longer to produce. Judging whether a compiled circuit is correct is hard on its own, an instance of the quantum oracle problem. The community also has no consolidated picture of how quantum compilers are tested, and even less of how they are regression-tested. This paper sets out a protocol for a Systematic Literature Review (SLR) of that area, following Kitchenham's guidelines and the PRISMA reporting standard. It states six research questions and their objectives, including one on which classical compiler-testing techniques have carried over to the quantum setting and which have not. It also sets out the databases to be searched, the search strategy, the inclusion, exclusion, and quality-assessment criteria, the data to be extracted, and how studies will be selected and synthesised. The goal is a reproducible map of the field that can ground later work on testing the quantum compiler, in a way that is both trustworthy and affordable. The protocol uses two independent reviewers with a measured level of agreement, a thematic plan for synthesising the evidence, and an explicit look at threats to validity.

Keywords: *Quantum software testing, Quantum compiler, Transpiler, Regression testing, Test oracle, Equivalence checking, Systematic Literature Review Protocol (SLRP), Qiskit.*

I. Introduction

Quantum computing has moved from theory to practice. Much of this progress relies on software development kits (SDKs) such as Qiskit, Cirq, and tket. Before a quantum program can run on real hardware, it must be rewritten to fit the machine's native gate set and limited qubit connectivity. In Qiskit [10], this rewriting is done by the transpiler, a staged compiler whose passes handle layout, routing, translation, optimisation, and scheduling. These SDKs evolve quickly, and the transpiler is one of their most frequently revised parts. A change to any pass can introduce a regression. The compiled circuit may compute a different result, carry more gates or greater depth, or simply take longer to produce. Catching such regressions is a testing problem, made harder by the quantum oracle problem: deciding whether a compiled circuit is correct is expensive. The black-box output-equivalence oracle the field usually relies on also cannot see every fault.

Testing of quantum software, and of quantum compilers specifically, is now an active area. Researchers have worked on metamorphic testing, differential testing, property-based testing, equivalence checking, and formal verification. But this work is scattered across different communities, and no one has yet drawn it together into a systematic view of how the quantum compiler is tested, or regression-tested. This paper lays out an SLR protocol, following Kitchenham [1] and PRISMA [2], to plan such a review rigorously and reproducibly.

The primary contributions of this protocol are:

- To identify and classify the testing techniques proposed for quantum software compilers.
- To analyse the correctness oracles used for compiled quantum circuits and their limitations.
- To assess how regression testing, including test selection and prioritisation, is addressed for quantum compilers.
- To trace the classical compiler-testing techniques these approaches derive from, and to identify which have no quantum counterpart.
- To catalogue the benchmarks, datasets, and tools used to evaluate quantum-compiler testing.
- To synthesise the open challenges and research gaps that motivate future work.

The rest of the paper is organised as follows. Section II gives background and related work. Section III presents the research method: research questions, objectives, data sources, search strategy, study selection, quality assessment, and data extraction. Section IV concludes.

II. Background

Quantum compilation turns an abstract, hardware-agnostic circuit into an equivalent circuit that a specific device can run. Layout assigns qubits, routing satisfies the coupling map, translation rewrites gates into the native basis, optimisation reduces gate count and depth, and scheduling places operations in time. The exact passes that run depend on the circuit, the backend, and the SDK version. Compilation can also permute qubits. Because of this, a compiled circuit equals the original only up to a qubit permutation and a global phase, which makes correctness checking subtle and expensive.

Deciding whether two circuits are equivalent is itself an instance of the quantum oracle problem. Comparing the full unitary matrices exactly does not scale past small circuits. So the field instead uses sampled, observable, metamorphic, or structural checks, which trade completeness for practicality. The most common check, a black-box output-equivalence oracle, looks only at the output. It can therefore pass a circuit whose internal compiler state has been corrupted, as long as the output still matches. Difficulties like these are what make a systematic look at compiler testing worthwhile.

Related work.

A number of research threads feed into this review. Empirical studies show that compiler and transpiler components are a significant source of defects in quantum platforms [3], [4]. Several tools already address one piece of the problem each. Metamorphic and differential testing tools include MorphQ [5], QDiff [7], and QSPE. Fuzzing and equivalence-modulo-inputs tools include QuteFuzz [11] and QEMI [12]. QuCheck [8] is a property-based tool, and equivalence-checking techniques are covered in [6]. Formal tools include Giallar [13] and CertiQ [14], and Bugs4Q [15] is a bug benchmark. Classical software engineering also has a mature literature on regression test selection and prioritisation, where cost-, history-, and change-aware heuristics are well-established baselines [9]. What no one has done yet is pull these threads together into one systematic view of the quantum compiler.

Existing systematic reviews.

We checked that a systematic review of this specific scope does not already exist. We searched the electronic databases for existing secondary studies using strings such as ("quantum compiler" OR "quantum transpiler") AND ("systematic literature review" OR "survey" OR "mapping study") AND (testing OR "regression testing"). Existing secondary studies address quantum software testing broadly [3], [21]. None of them focuses specifically on testing and regression testing of the quantum compiler. This confirms the need for this review.

III. Research Method

The research method follows the SLR guidelines of Kitchenham [1]. It covers the research questions (Table 1), the research objectives (Table 2), the electronic data sources (Table 3), and the search strategy and search terms (Table 4). It also covers the database-specific queries and the scoping search used to validate them (Appendix D, Table 5; Appendix C, Table 11), the study-selection process with its inclusion/exclusion criteria (Table 6) and flow (Figure 1), the quality-assessment checklist (Table 7), and the data-extraction form (Table 8).

Protocol Amendments.

Fixing the questions, the search, and the selection and analysis rules in advance matters. It lets a reader later check the finished review against this plan, and see where, if at all, the two differ. If anything changes while the review is under way, for example a search string, an inclusion rule, or an extraction item, we will log the change with its date and reason, and report it in the final paper.

Research Questions.

The review is organised around six research questions, shown in Table 1.

RQ5 touches on a body of work this review does not treat as primary evidence. Most quantum-compiler testing techniques started out in classical software testing and were later adapted to quantum circuits. This includes metamorphic testing, differential testing, equivalence modulo inputs [22], random program generation, property-based testing, translation validation, and compiler verification. This classical body of work is large, and Chen et al. [23] have already surveyed it in full, so we exclude it as primary evidence and use it only as background. Instead, we capture the connection at extraction: each included study is mapped to the classical technique it comes from (Table 8). This lets RQ5 report not just what transferred to the quantum setting, but what did not. A review of quantum papers alone could not show that second part. A claim that something did not transfer needs a fixed list of candidates decided before the search begins. Table 10 (Appendix B) pre-declares eight such classical technique families, each with its original source, checked against Chen et al.’s survey [23]. If a family has no quantum descendant among the included studies, we report it as an empty row. Large-language-model-guided test generation has no single classical ancestor of its own, so we include it under RQ1 instead, and record it as a modifier at extraction rather than as a ninth pre-declared family.

Table 1: Research Questions

ID	Research Question	Motivation
RQ1	What testing techniques have been proposed for quantum software compilers (transpilers)?	To identify and classify the landscape of quantum-compiler testing approaches.
RQ2	What correctness oracles are used for compiled quantum circuits, and what are their limitations?	To understand how correctness is decided and where the oracles fall short.
RQ3	How is regression testing, including test selection and prioritisation, addressed for quantum compilers?	To determine whether budgeted regression testing exists for the compiler.
RQ4	What benchmarks, datasets, and tools support the evaluation of quantum-compiler testing?	To catalogue the evaluation infrastructure available to researchers.
RQ5	Which classical compiler-testing and regression-testing techniques have been transferred to the quantum compiler, how were they adapted, and which have not been transferred?	To trace the classical antecedent of each quantum technique and expose which parts of the classical toolkit have no quantum counterpart.
RQ6	What are the open challenges and research gaps in testing quantum compilers?	To surface directions for future research.

Research Objectives.

The objectives, mapped one-to-one to the research questions, are shown in Table 2.

Table 2: Research Objectives

ID	Objective (answers)
O1	To catalogue and classify the testing techniques proposed for quantum compilers. (RQ1)
O2	To analyse correctness-oracle strategies for compiled circuits and their limitations. (RQ2)
O3	To assess regression-testing, selection, and prioritisation methods for quantum compilers. (RQ3)

O4	To identify the benchmarks, datasets, and tools used for evaluation. (RQ4)
O5	To trace the classical antecedents of quantum-compiler testing techniques and identify which classical techniques have no quantum counterpart. (RQ5)
O6	To synthesise the open challenges and research gaps in the area. (RQ6)

Data Sources.

The primary sources are the electronic databases listed in Table 3, chosen for their coverage of software engineering and quantum-computing venues. arXiv captures recent preprints and grey literature.

Table 3: Data Sources

Source	URL
IEEE Xplore	https://ieeexplore.ieee.org
ACM Digital Library	https://dl.acm.org
ScienceDirect (Elsevier)	https://www.sciencedirect.com
SpringerLink	https://link.springer.com
arXiv	https://arxiv.org

Search Strategy.

The search covers 2015-2026. This start date sits roughly two years ahead of Qiskit's 2017 release [10], the SDK that most compiler-testing work targets. The scoping search (Appendix C) confirms this date empirically: the same search terms, run against arXiv records from before 2015, return no results. The window includes journals, conferences, symposiums, workshops, and peer-reviewed preprints. Search terms are grouped by concept (Table 4). We combine concepts with Boolean AND, and terms within each concept with OR, giving the representative string: ("quantum" AND ("transpiler" OR "quantum compiler" OR "compilation")) AND (testing OR bug OR fault OR oracle OR "equivalence checking" OR "regression testing" OR "test selection").

Table 4: Search Terms by Concept

Concept	Search terms (combined with OR)
Quantum software	"quantum software", "quantum program", "quantum SDK", Qiskit, Cirq, tket, PyQuil
Compiler / transpiler	transpiler, "quantum compiler", compilation, "pass manager", "qubit mapping", routing
Testing / faults	testing, test, bug, fault, defect, miscompilation, regression
Oracle / technique	oracle, "equivalence checking", metamorphic, "differential testing", "property-based", mutation
Selection	"test selection", "test prioritisation", "regression test", cost-aware

Search Procedure.

We turn the concept terms of Table 4 into a database-specific query for each of the five sources. Appendix D (Table 5) gives the full, re-runnable query for every database, written in that engine's own field-tag syntax, along with its field scope, date restriction, and any workaround needed where a database limits query length or Boolean nesting. SpringerLink, for example, requires the query to be split into pooled sub-queries. We also apply backward and forward snowballing, following Wohlin's guidelines [19], to the reference lists and citing papers of every included primary study. This recovers relevant work that the search terms alone do not index.

Scoping Search.

Before fixing the search strings above, we ran a scoping search. This tested whether the concept terms of Table 4 retrieve the target literature, estimated how much work the review will involve, and surfaced problems while they were still cheap to fix. This follows Kitchenham's recommendation [1] for a validation run before registration. We drew counts from the arXiv API, OpenAlex, and DBLP, and, where available without institutional sign-in, from ACM Digital Library, ScienceDirect, IEEE Xplore, and SpringerLink. Full figures are in Table 11 (Appendix C). We made four checks. First, known-item retrieval: eight seed studies already known to be in scope, MorphQ [5], QDiff [7], QuCheck [8], QuteFuzz [11], QEMI [12], Giallar [13], CertiQ [14], and Bugs4Q [15], were all returned by the concept terms of Table 4 alone. Second, volume and the date boundary: the full concept string returned 116 arXiv and 215 OpenAlex records for 2015-2026, a manageable yield for two reviewers within the six-month schedule set out below. The same search restricted to submissions before 2015 returned zero records, which confirms the window's lower bound empirically rather than by argument alone. Third, precision: the terms "test selection" and "test prioritisation" combined with "quantum" surfaced a systematic false-positive class, namely quantum optimisation algorithms applied to classical test-suite optimisation, the mirror image of this review's subject. An explicit exclusion criterion (Table 6) now removes this class at the title-and-abstract stage. Fourth, the gap and competing studies: compiler or transpiler terms combined with the exact phrase "regression testing" returned a single arXiv record for the whole period, CertiQ [14]. That paper only notes in passing that its verified passes still satisfy Qiskit's existing test suite, and neither proposes nor evaluates a regression-testing method. The closest secondary studies address quantum software testing broadly [3], [21], but do not analyse compiler testing as such, which shows that RQ3 addresses a real gap rather than an assumed one. Consistent with Kitchenham's guidance, we report the scoping search here only as a design step. It calibrates the search strings, the time window, and the criteria. It does not contribute any included studies. All of those will come from the formal search of Table 5 (Appendix D).

Study Selection.

We select primary studies by applying the inclusion and exclusion criteria of Table 6. We look first at the title, abstract, and conclusion of each candidate, then at the full text. The study-selection process follows the PRISMA stages of identification, screening, eligibility, and inclusion, shown in Figure 1. Figure 1 also records the exclusion reasons at both screening stages, so the reduction from the broad search results to the final included set stays transparent.

Table 6: Inclusion / Exclusion Criteria

Inclusion criteria	Exclusion criteria
Peer-reviewed papers or citable preprints on testing or analysis of quantum compilers or transpilers.	Duplicate records of the same study across databases.
Studies proposing or evaluating a testing technique, oracle, or regression-testing method for quantum compilation.	Papers not available in the English language.
Studies published between 2015 and 2026.	Papers on a different subject (for example, pure quantum algorithms or hardware) not concerned with compiler testing.
Empirical or methodological studies with a described approach and evaluation.	Short abstracts, posters, or editorials without a described method.
Studies whose subject is the quantum compiler or transpiler itself, that is, quantum software under test.	Studies that apply quantum computing or quantum-inspired optimisation (for example QAOA or quantum-behaved particle swarm optimisation) to classical test selection, prioritisation, or minimisation. These use quantum methods to perform testing rather than testing a quantum compiler, and are out of scope. This criterion was added following the scoping search, which showed the class to be a frequent false positive.

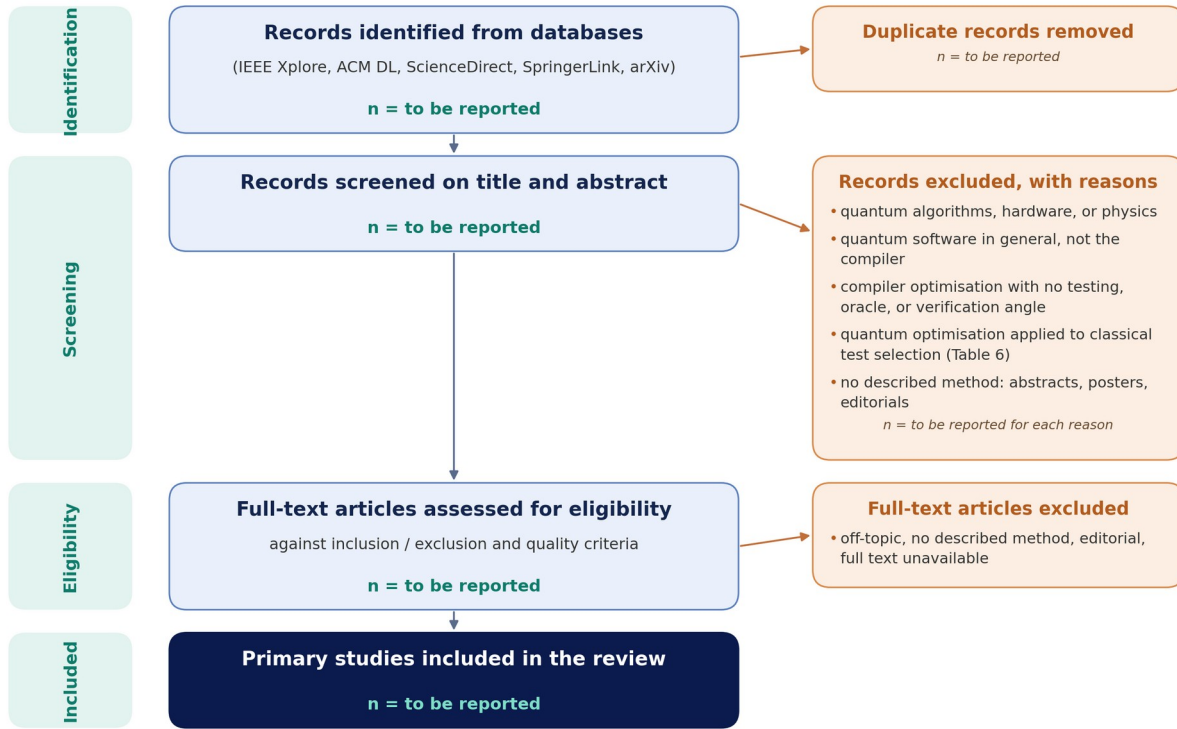


Figure 1: Primary studies selection process (PRISMA-based).

The search is deliberately broad, so a steep drop from the initial hits to the final set is expected. At the title-and-abstract stage, we set records aside for five reasons. First, the record is about quantum algorithms, hardware, or physics. Second, it is about quantum software in general rather than the compiler. Third, it is about compiler optimisation with no testing, oracle, or verification angle. Fourth, it uses quantum methods to perform classical testing, the class identified by the scoping search. Fifth, it reports no described method. Figure 1 lists these five reasons and carries a count against each.

Selection Procedure and Reliability.

We import every record from the databases into a reference manager, Zotero [17], where we remove exact and near-duplicate entries before screening. Screening itself runs in Rayyan [16]. Two reviewers then work through the titles and abstracts, and later the full texts, independently, against the criteria in Table 6. We measure inter-rater agreement with Cohen's kappa [20]. Screening continues only once agreement reaches at least 0.6, the usual threshold for substantial agreement. We talk through any disagreements, and where the two reviewers cannot agree, the third author adjudicates. Because this adjudication covers only the disputed cases, we report it separately as a resolution step rather than folding it into the agreement statistic. Before full extraction, both reviewers pilot the form in Table 8 on a random handful of included

studies, so that each item is read the same way, and the rest of the extraction is then cross-checked.

Quality Assessment.

Both reviewers score each eligible study independently against the six checklist items of Table 7 (QA1-QA6). Each item scores 1 (yes), 0.5 (partly), or 0 (no), for a maximum of 6. Studies scoring below the pre-declared threshold of 4.0 stay in the synthesis, but we flag them as lower-quality evidence, and a sensitivity analysis re-runs the synthesis without them. We chose to retain rather than exclude these studies because the field is young, and a strict cut-off would discard early tool papers that carry real information about technique coverage for RQ1 and RQ4. So quality weights our confidence in a finding, rather than gating entry, consistent with Kitchenham's guidance [1]. Quality scoring follows the same reliability process as screening: independent scoring, Cohen's kappa [20], and disagreements resolved by discussion or, failing that, by the third author. A one-grade disagreement is resolved to the lower score, and only disputes that cross the 4.0 threshold are escalated. The one exception is QA2: if a study's technique or oracle cannot be determined at all, we exclude it, since we cannot classify it for any research question.

Table 7: Quality Assessment Checklist

ID	Quality assessment question	Score
QA1	Are the aims and scope of the study clearly stated?	0 / 0.5 / 1
QA2	Is the testing technique or oracle clearly described?	0 / 0.5 / 1
QA3	Is the evaluation methodology sound (benchmarks, metrics, baselines)?	0 / 0.5 / 1
QA4	Are the results clearly reported and supported by evidence?	0 / 0.5 / 1
QA5	Are the limitations and threats to validity discussed?	0 / 0.5 / 1
QA6	Are the tools, data, or artifacts available for reproduction?	0 / 0.5 / 1

Data Extraction.

For every included study, the form in Table 8 records the testing technique, the correctness oracle and the fault classes it can and cannot see, how far the study relates to regression testing, the benchmarks and tools involved, and the open problems the authors report. Using one form for every study is what makes the extracted data comparable at synthesis. The lead author extracts data for every included study, and the second reviewer independently checks it in full against the source paper. Three items call for judgement rather than plain transcription: oracle limitations, regression-testing relevance, and the classical antecedent. Both reviewers extract these three independently for every study, and resolve disagreements by discussion or, failing that, by the third author. The classical antecedent belongs in this group because tracing a quantum technique back to its classical ancestor is an inference, not a fact stated on the page, and the negative finding under RQ5 depends on more than one reader making these judgements consistently. Where a paper does not report an item, we record the field as not reported and contact the corresponding authors once, giving them four weeks to reply before it stays as not reported. The technique, the oracle and its limitations, and regression-testing relevance are the

primary items, and they answer RQ1-RQ3 directly. The classical antecedent is the primary item for RQ5. The benchmarks, tools, and reported open challenges are secondary, feeding RQ4 and RQ6.

Table 8: Data-Extraction Form (Data-Item Description)

Data item	Description	RQ(s)
Study metadata	Title, authors, publication venue, year, and source database	RQ1
Testing technique	The testing or analysis technique proposed or evaluated (differential, metamorphic, fuzzing, equivalence modulo inputs, formal verification, or benchmark)	RQ1
Correctness oracle	The oracle used to decide correctness and how it is computed	RQ2
Oracle limitations	The fault classes the oracle can and cannot observe	RQ2
Regression-testing relevance	Whether and how the study supports regression testing, test selection, or prioritisation	RQ3
Benchmarks and tools	Benchmarks, datasets, subject compilers, and tool or artifact availability	RQ4
Classical antecedent	The classical compiler-testing or regression-testing technique the approach derives from (for example metamorphic testing, differential testing, equivalence modulo inputs, random program generation, property-based testing, translation validation, compiler verification, or regression test selection and prioritisation), together with what the authors had to change to make it work on quantum circuits, and any classical assumption they report as no longer holding	RQ5
Open challenges	Limitations and open problems the study reports	RQ6

Data Synthesis.

The included studies differ too much in technique, oracle, target compiler, and evaluation for a meta-analysis to make sense. So we draw the evidence together qualitatively instead. We do not define any criterion for quantitative synthesis, and we do not plan any summary measures, data-combination methods, or consistency statistics such as I-squared or Kendall's tau. This is a deliberate design decision, not an omission. Our approach is thematic synthesis in the sense of Cruzes and Dyba [18]: we code the extracted text, gather the codes into themes that line up with the research questions, and compare the themes across studies. Cross-tabulations carry most of the weight. One classifies each study by technique and answers RQ1. A second sets each study's oracle against the fault classes it can and cannot observe, answering RQ2. A third traces each study to the classical technique it adapts, answering RQ5. Classical techniques with no quantum descendant anywhere in the included set appear as the empty rows of that table. A fourth maps technique against oracle against research question, addressing RQ3, RQ4, and RQ6. Around these tables, the review states, question by question, what the evidence settles, where it disagrees, and what stays open. Because we fix the method before running the search, the eventual findings cannot be quietly shaped to fit the data.

Threats to Validity.

This protocol, and the review it will guide, face the usual threats. The search may miss papers that use vocabulary we did not expect. We try to keep this loss small with concept-based search strings that include synonyms, five databases with their own detailed queries (Appendix D), and forward and backward snowballing. We also ran the scoping search partly to expose this risk before the review begins.

A related threat affects precision rather than recall. The scoping search also retrieves work that applies quantum optimisation algorithms to classical test-selection problems, which is the reverse of our subject. An explicit exclusion criterion (Table 6) now targets this class of paper.

Restricting eligibility to English-language reports is a further limitation. This is practical, not principled: the review team cannot screen reliably in other languages. The risk is smaller than it might seem, because the venues that publish quantum-compiler testing work are overwhelmingly English-language. Any non-English work we find during snowballing will still be listed, so its exclusion stays visible.

Screening is a matter of judgement. So two reviewers work independently, we hold agreement to at least 0.6 by Cohen's kappa [20], and the third author adjudicates any remaining disputes. That adjudicator is a co-author rather than an uninvolved third party, since the review team is a doctoral candidate and two supervisors. We handle this limitation through procedure and disclosure rather than pretending it away: every screening decision, including disputed records and how each was resolved, is exported from Rayyan and kept by the author team for direct inspection.

There is also the familiar bias that striking or tool-backed results are more likely to get published, made worse by databases that resist automated searching. Including grey literature and arXiv preprints, and searching by hand where automation fails, is meant to stop this from skewing what we find.

The negative half of RQ5 carries a sharper threat. A single study we fail to retrieve, one that adapts regression test selection to a quantum compiler, would disprove a reported non-transfer. Three measures guard against this. The frame of Table 10 is fixed in advance, so the claim only covers a closed, stated set of technique families. Backward and forward snowballing reaches work the concept terms miss. And before we report any family as having no quantum descendant, we run a targeted verification search that pairs its canonical vocabulary with the quantum-compiler terms. So the finding is stated as absence of evidence within a defined, reproducible frame, not as evidence of absence.

Classifying techniques and oracle limitations again calls for judgement, which the piloted form and cross-checking are there to contain. Finally, because this is a qualitative mapping of techniques rather than a meta-analysis of interventions, a formal GRADE rating would not fit. Instead, we state our confidence in each finding in plain terms, based on how many independent studies support it and how well they score on quality (Table 7).

Review Team, Timeline, and Dissemination.

The work is shared across the author team. The lead author designs the protocol and carries out the searching, screening, extraction, and synthesis. The two co-authors serve as the second reviewer and adjudicator, and provide methodological oversight. We expect the full review to take about six months. The first month finalises the protocol and runs the search across all five databases. The next two months cover screening and full-text selection. Two more months cover quality assessment, extraction, and synthesis. The final month is for writing and internal review. The finished review will go to a peer-reviewed software-engineering venue, and we will make the search exports and extraction records available so others can replicate the work.

IV. Conclusion and Future Work

This paper has set out a protocol for a systematic review of testing and regression testing in quantum software compilers. It is built on Kitchenham's guidelines and the PRISMA standard. The protocol fixes the research questions and objectives, the data sources and search strategy, the selection and quality criteria, the extraction plan, the synthesis method, and the threats to validity that a rigorous, reproducible review needs. Once carried out, the review should produce five things: a classified map of compiler-testing techniques, an account of the correctness oracles in use and where they fall short, a catalogue of the available benchmarks and tools, a lineage tracing each quantum technique to the classical method it adapts and the classical methods with no quantum counterpart, and a clear statement of what remains open. These results are meant to form the evidence base for further work on testing the quantum compiler, in a way that is both trustworthy and affordable.

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Author Contributions. F.N. designed the protocol, defined the research questions and search strategy, and drafted the manuscript. A.S. and I.A. contributed to the methodology, acted as second reviewer and adjudicator for study selection, and critically revised the manuscript. All authors read and approved the final protocol.

Data Availability. No primary data were generated for this protocol. The full database-specific search strings of Table 5, the screening decisions, and the completed data-extraction records will be made available upon reasonable request once the review is carried out.

References

- [1] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Technical Report EBSE-2007-01, Keele University and Durham University, 2007.
- [2] M. J. Page et al., "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, n71, 2021.

- [3] M. Paltenghi and M. Pradel, "A survey on testing and analysis of quantum software," arXiv:2410.00650, 2024.
- [4] M. Paltenghi and M. Pradel, "Bugs in quantum computing platforms: an empirical study," Proc. ACM Program. Lang. (OOPSLA), 2022.
- [5] M. Paltenghi and M. Pradel, "MorphQ: metamorphic testing of the Qiskit quantum computing platform," in Proc. ICSE, 2023.
- [6] L. Burgholzer and R. Wille, "Advanced equivalence checking for quantum circuits," IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst., vol. 40, no. 9, pp. 1810–1824, 2021.
- [7] J. Wang, Q. Zhang, G. H. Xu, and M. Kim, "QDiff: differential testing of quantum software stacks," in Proc. IEEE/ACM ASE, 2021.
- [8] G. Pontolillo et al., "QuCheck: a property-based testing framework for quantum programs in Qiskit," arXiv:2503.22641, 2025.
- [9] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritisation: a survey," Softw. Test. Verif. Reliab., vol. 22, no. 2, pp. 67–120, 2012.
- [10] A. Javadi-Abhari et al., "Quantum computing with Qiskit," arXiv:2405.08810, 2024.
- [11] I. Iwumbwe, B. Z. Liu, and J. Wickerson, "QuteFuzz: fuzzing quantum compilers using randomly generated circuits with control flow and subcircuits," in Proc. PLanQC, 2025.
- [12] J. Luo, S. Xia, F. Zhang, and J. Zhao, "QEMI: a quantum software stacks testing framework via equivalence modulo inputs," arXiv:2602.09942, 2026.
- [13] R. Tao et al., "Giallar: push-button verification for the Qiskit quantum compiler," in Proc. PLDI, 2022.
- [14] Y. Shi et al., "CertiQ: a mostly-automated verification of a realistic quantum compiler," arXiv:1908.08963, 2020.
- [15] P. Zhao, J. Zhao, Z. Miao, and S. Lan, "Bugs4Q: a benchmark of real bugs for quantum programs," in Proc. ASE-NIER, 2021.
- [16] M. Ouzzani, H. Hammady, Z. Fedorowicz, and A. Elmagarmid, "Rayyan: a web and mobile app for systematic reviews," Systematic Reviews, vol. 5, art. 210, 2016.
- [17] Corporation for Digital Scholarship, "Zotero (reference management software)," 2024. [Online]. Available: <https://www.zotero.org>
- [18] D. S. Cruzes and T. Dyba, "Recommended steps for thematic synthesis in software engineering," in Proc. Int. Symp. Empirical Software Engineering and Measurement (ESEM), 2011, pp. 275-284.
- [19] C. Wohlin, "Guidelines for snowballing in systematic literature studies and a replication in software engineering," in Proc. Int. Conf. Evaluation and Assessment in Software Engineering (EASE), 2014, pp. 1-10.

- [20] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37-46, 1960.
- [21] M. De Stefano, F. Pecorelli, D. Di Nucci, F. Palomba, and A. De Lucia, "The quantum frontier of software engineering: a systematic mapping study," *Information and Software Technology*, vol. 175, art. 107525, 2024.
- [22] V. Le, M. Afshari, and Z. Su, "Compiler validation via equivalence modulo inputs," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2014, pp. 216-226.
- [23] J. Chen, J. Patra, M. Pradel, Y. Xiong, H. Zhang, D. Hao, and L. Zhang, "A survey of compiler testing," *ACM Computing Surveys*, vol. 53, no. 1, art. 4, 2020.
- [24] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, 2011, pp. 283-294.
- [25] W. M. McKeeman, "Differential testing for software," *Digital Technical Journal*, vol. 10, no. 1, pp. 100-107, 1998.
- [26] T. Y. Chen, S. C. Cheung, and S. M. Yiu, "Metamorphic testing: a new approach for generating next test cases," *Dept. of Computer Science, Hong Kong University of Science and Technology*, Tech. Rep. HKUST-CS98-01, 1998.
- [27] K. Claessen and J. Hughes, "QuickCheck: a lightweight tool for random testing of Haskell programs," in *Proc. ACM SIGPLAN Int. Conf. Functional Programming (ICFP)*, 2000, pp. 268-279.
- [28] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Proc. Int. Conf. Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1998, pp. 151-166.
- [29] X. Leroy, "Formal verification of a realistic compiler," *Communications of the ACM*, vol. 52, no. 7, pp. 107-115, 2009.

Appendix A. PRISMA-P 2015 Compliance

Table 9 maps all 26 reporting points of the PRISMA-P 2015 checklist, that is the 17 numbered items together with their sub-items, to the section of this protocol where each is addressed. Items that do not apply to a technique-mapping review are listed with the reason rather than omitted.

Table 9: PRISMA-P 2015 checklist mapping

PRISMA-P item	Where addressed in this protocol
1a Title – identification	Title, identified in the title and abstract as a systematic review protocol.
1b Title – update	Not applicable: this is a protocol for a new review, not an update.
2 Registration	Protocol Amendments. This protocol is not registered in a public registry. It is submitted for supervisory review, with the fixed research questions, search strategy, and selection criteria below serving the same advance-commitment function that registration would.
3a Authors – contact	Author block: names, institutional affiliation, and corresponding-author contact details.
3b Authors – contributions	Author Contributions. F.N. is the guarantor of the review.
4 Amendments	Protocol Amendments: any change to the questions, search, criteria, or extraction items will be logged with its date and reason, and reported in the final review.
5a Support – sources	Funding: no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.
5b Support – sponsor	Funding: no external sponsor. The review is conducted as part of doctoral research at City University of Science & Information Technology, Peshawar.
5c Support – role of sponsor or funder	Funding: as there is no funder or sponsor, no external party had any role in designing this protocol. The institution provided no input beyond doctoral supervision, which is recorded under Author Contributions.
6 Rationale	Section I, Introduction; Section II, Background and Related Work, including the absence of an existing secondary study of this scope.
7 Objectives	Research Questions (Table 1, six questions) and Research Objectives (Table 2, mapped one-to-one). PICO does not directly apply to a technique-mapping review of software artefacts. Instead, the questions are framed over population (quantum compilers and transpilers), intervention (testing techniques and correctness oracles), comparison (across technique classes, across oracle capability, and against the classical antecedent each technique adapts), and outcome (fault classes observable, regression-testing support, and reported open challenges). The scope boundary excluding classical studies as primary evidence is stated under Research Questions.
8 Eligibility criteria	Inclusion / Exclusion Criteria (Table 6). Study period 2015–2026 justified under Search Strategy. English-language and peer-reviewed or citable-preprint restrictions stated in Table 6.
9 Information sources	Data Sources (Table 3): five electronic databases including arXiv for grey literature, with coverage dates 2015–2026, supplemented by backward and forward snowballing (Search Procedure).
10 Search strategy	Search Strategy (Table 4, concept terms) sets out the terms. Search Procedure and Appendix D (Table 5) give the complete, re-runnable query for all five databases, not just one, with the limits each database imposes. The Scoping

	Search subsection reports how these strings were validated, including arXiv yields, known-item recall against eight seed studies, and an empirical check of the 2015 lower bound.
11a Data management	Selection Procedure and Reliability: records managed in Zotero [17] for de-duplication and in Rayyan [16] for screening. Extraction records retained by the author team.
11b Selection process	Selection Procedure and Reliability: two reviewers screen titles, abstracts, and full texts independently against Table 6. Cohen's kappa [20] applies with a 0.6 threshold. Third-author adjudication resolves residual disputes. PRISMA flow is shown in Figure 1.
11c Data collection process	Data Extraction: a single extraction form (Table 8), piloted independently by both reviewers on a random subset. The lead author extracts factual items, and the second reviewer verifies them in full against the source. Two interpretive items, oracle limitations and regression-testing relevance, are extracted independently in duplicate by both reviewers. The classical antecedent is a third item extracted the same way, independently and in duplicate. Unreported items are recorded as not reported, and authors are contacted once, with a four-week window to reply.
12 Data items	Data Extraction (Table 8): study metadata, testing technique, correctness oracle, oracle limitations, regression-testing relevance, classical antecedent and what had to change, benchmarks and tools, and open challenges, each mapped to the research question it serves. The candidate set of classical technique families for the antecedent item is pre-declared in Table 10 (Appendix B).
13 Outcomes and prioritization	Data Extraction: the primary items are the technique, the oracle and its limitations, regression-testing relevance, and the classical antecedent. Together they answer RQ1–RQ3 and RQ5. Benchmarks, tools, and reported open challenges are secondary items, feeding RQ4 and RQ6.
14 Risk of bias in individual studies	Quality Assessment (Table 7, QA1–QA6) is assessed at the study level. Both reviewers score independently, and we report Cohen's kappa [20]. Ordinal disagreements are resolved to the lower score, and only disputes that cross the 4.0 threshold go to the third author. Studies scoring below 4.0 are retained but flagged. Quality then feeds into synthesis by weighting our confidence in each finding, backed by a sensitivity analysis that excludes low-scoring studies. QA2 is the sole basis for exclusion.
15a Data synthesis – criteria for quantitative synthesis	Data Synthesis: quantitative synthesis is judged inappropriate in advance. The included studies differ too much in technique, oracle, target compiler, and evaluation design, and report no common effect measure. No criterion for meta-analysis can be satisfied.
15b Data synthesis – summary measures and combination	Not applicable: no meta-analysis is planned, so no summary measures, methods of combining data, or consistency statistics (I^2 , Kendall's τ) are specified. Stated explicitly under Data Synthesis.
15c Data synthesis – additional analyses	Quality Assessment: a sensitivity analysis re-runs the synthesis excluding studies scoring below 4.0. Subgroup

	comparison is by technique class and by oracle type, as described under Data Synthesis.
15d Data synthesis – type of summary if not quantitative	Data Synthesis: thematic synthesis in the sense of Cruzes and Dybå [18]. The results are reported as four cross-tabulations: one by technique, one setting oracle against observable fault class, one setting classical antecedent against quantum descendant (where an empty row is itself a finding), and one setting technique against oracle against research question. Each is paired with a narrative answer to the relevant research question.
16 Meta-bias(es)	Threats to Validity covers several kinds of bias. Publication bias is addressed by including grey literature and arXiv preprints. Tooling bias, from databases that resist automated querying, is addressed by manual searching. Language bias from the English-only restriction, and the limited independence of a co-author adjudicator, are both disclosed, with screening decisions retained for inspection. Selective-reporting risk within studies is captured by QA4 and QA6. A further risk is that RQ5 reports a non-transfer that a missed study would contradict. This is addressed by the pre-declared frame of Table 10, by snowballing, and by a targeted per-family verification search.
17 Confidence in cumulative evidence	Threats to Validity: GRADE does not apply here, since this is a qualitative mapping of techniques rather than a synthesis of intervention effects. Instead, confidence in each finding is stated narratively, based on the number of independent supporting studies and their quality scores (Table 7).

Appendix B. Pre-Declared Classical Technique Frame for RQ5

Table 10 fixes the candidate set of classical technique families against which RQ5 judges transfer and non-transfer. The frame is assembled from the canonical origin of each family and was fixed before the formal search was run.

Table 10: Pre-declared classical technique families (RQ5 reference frame)

ID	Classical technique family	Canonical origin
C1	Randomised and grammar-based program generation	Yang et al. [24]
C2	Differential testing	McKeeman [25]
C3	Metamorphic testing	Chen, Cheung & Yiu [26]
C4	Equivalence modulo inputs	Le, Afshari & Su [22]
C5	Property-based testing	Claessen & Hughes [27]
C6	Translation validation	Pnueli, Siegel & Singerman [28]
C7	Proof-based compiler verification	Leroy [29]
C8	Regression test selection and prioritisation	Yoo & Harman [9]

Appendix C. Scoping Search Yield

Table 11 records the yield of the scoping search across the three open, scriptable bibliographic sources, plus IEEE Xplore and SpringerLink, both of which turned out to expose a total-hit count on their public search results page without requiring institutional sign-in. All counts cover 2015–2026 and were retrieved programmatically or via direct query where retrieved from a public results page so that they can be reproduced exactly. They are single-pass scoping figures used to validate the search terms and gauge feasibility. They are not the final corpus, which will come from the full search of Table 5 across all five databases with de-duplication and snowballing.

Table 11: Scoping search yield by query and source (2015–2026)

Query (concept)	arXiv	OpenAlex	DBLP	ACM DL	Sci Direct	IEEE Xplore	SpringerLink
Full concept string: quantum × (transpiler / compiler) × (testing / oracle / ...)	116	215	—	304	2	52	161
“quantum compiler” × testing	19	42	3	—	—	23	—
quantum transpiler × testing	58	55	8	—	—	0	—
quantum × equivalence checking	51	122	—	—	—	34	—
compiler / transpiler × “regression testing” (RQ3/RQ5 gap probe)	1	6	—	43	0	0	2
quantum × (“test selection” / “prioritisation”) (confound, excluded)	2	12	—	—	—	6	—
control: “transpiler” × testing with NO quantum term (JS/Babel noise)	—	130	—	—	—	—	—
pre-2015 boundary check: compiler × testing before 2015	0	—	—	—	—	—	—

Dashes denote queries not issued against that source (for example DBLP indexes titles only, so broad conceptual strings are not meaningful there). ACM Digital Library counts were retrieved from a signed-in session using the Table 5 query. They cover the whole record, not just the title and abstract, because the AllField scope includes full text and reference lists. Because of this wider scope, ACM counts run higher than, and are not directly comparable to, the title-and-abstract counts of arXiv, OpenAlex, and DBLP. In particular, the 43 ACM records for the regression-testing probe are mostly papers that merely cite regression-testing work in their bibliographies. Very few are actually about regression testing of a quantum compiler. This is why the gap finding rests on the title-and-abstract sources instead. ScienceDirect counts were retrieved from a signed-in session with the title-abstract-keywords scope, so they are directly

comparable to arXiv, OpenAlex, and DBLP. Elsevier indexes comparatively few quantum-software and software-engineering venues, which is why its yield is low. IEEE Xplore counts were retrieved unauthenticated from its public Command Search results page, using the exact Table 5 query (Document Title/Abstract scope for “quantum”, All Metadata scope for the remaining terms). This makes them close in comparability to the ACM and ScienceDirect counts. Notably, the exact phrase “quantum transpiler” returns zero IEEE Xplore hits combined with any testing term, because IEEE-indexed venues (chiefly DAC/ASP-DAC/QCE) favour “compiler” over “transpiler”. The regression-testing gap probe is also zero for IEEE Xplore, reinforcing the same null result seen in arXiv and ScienceDirect. SpringerLink counts were retrieved unauthenticated from its public search results page. However, SpringerLink's search does not support field-scoped Boolean matching the way IEEE Xplore and ACM do. A manual check of its 2-record regression-testing-probe yield found one off-topic hit, a JavaScript side-channel paper. So the SpringerLink column is reported only as a rough, noisy upper bound, not a count comparable to the others, and was not pursued beyond the full-concept-string and regression-testing-probe rows. OpenAlex approximates the cross-database yield because it indexes the same publisher venues. The “no quantum term” control and the pre-2015 check are diagnostic only. They are not part of the review search.

Appendix D. Database-Specific Search Queries

Table 5 gives the full, re-runnable query issued to each of the five databases, in that engine's own field-tag syntax, together with the field scope and any limit on query length or Boolean nesting. All queries are applied to the title, abstract, and keyword or index-term fields and restricted to 2015-2026, with the workaround used for SpringerLink noted against that row.

Table 5: Database-Specific Search Queries

Database	Query as specified (fields; limits)
IEEE Xplore	("Document Title":quantum OR "Abstract":quantum) AND ("All Metadata":transpiler OR "All Metadata":"quantum compiler" OR "All Metadata":"pass manager" OR "All Metadata":"qubit mapping") AND ("All Metadata":testing OR "All Metadata":bug OR "All Metadata":fault OR "All Metadata":oracle OR "All Metadata":"equivalence checking" OR "All Metadata":"regression testing" OR "All Metadata":"test selection") — Command Search; 2015–2026.
ACM Digital Library	Title:(quantum) OR Abstract:(quantum) AND AllField: ("transpiler" OR "quantum compiler" OR "pass manager" OR "qubit mapping") AND AllField:("testing" OR "bug" OR "fault" OR "oracle" OR "equivalence checking" OR "regression testing" OR "test selection") — Advanced Search, "Search Within: Anywhere"; 2015–2026.
ScienceDirect	Title, abstract, keywords: quantum AND (transpiler OR "quantum compiler" OR "quantum compilation") AND (testing OR bug OR fault OR oracle OR "equivalence

	checking" OR "regression testing") — ScienceDirect caps Boolean connectors at 8 per field, so the oracle/selection terms are issued as a second pooled query; 2015–2026.
SpringerLink	(quantum AND (transpiler OR "quantum compiler")) AND (testing OR bug OR oracle OR "equivalence checking" OR "regression testing") — issued as two sub-queries (compiler-terms × testing-terms and compiler-terms × oracle-terms), results pooled and de-duplicated; date facet 2015–2026.
arXiv	all:(quantum AND (transpiler OR "quantum compiler" OR "quantum compilation")) AND all:(testing OR bug OR fault OR oracle OR "equivalence checking" OR "regression testing" OR "test selection") — issued through the arXiv API (categories cs.SE, cs.PL, quant-ph); submitted 2015-01-01 to 2026-12-31.
Canonical concept string	("quantum") AND ("transpiler" OR "quantum compiler" OR "quantum compilation" OR "pass manager" OR "qubit mapping" OR "circuit routing") AND ("testing" OR "test" OR "bug" OR "fault" OR "miscompilation" OR "oracle" OR "equivalence checking" OR "regression testing" OR "test selection" OR "test prioritisation" OR "test prioritization") — the database-independent form from which each query above is derived.